CS240: Database Systems

Lecture 04:

Relational Algebra and Calculus

Outline

- **Relational Algebra**
  - **Unary Relational Operations**
  - Binary Relational Operations
  - Additional Relational Operations
- Introduction to Relational Calculus
  - Tuple Relational Calculus
  - Domain Relational Calculus

Relational Query Languages

- Query Languages (QL): Allow manipulation and retrieval of data from a database
- Two mathematical Query Languages form the basis for "real" languages (e.g. SQL), and for implementation:
  - *Relational Algebra*: More operational (procedural), very useful for representing execution plans
  - *Relational Calculus*: Lets users describe what they want, rather than how to compute it: Non-operational, declarative

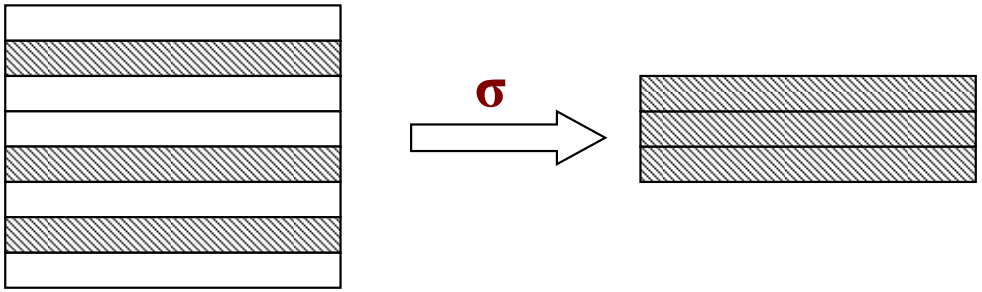
Relational Algebra

- *Relational algebra* is the basic set of *operations* for the relational model
- These operations enable a user to specify basic retrieval requests (*queries in terms of operators*)
- Every operator in relational algebra accepts (one or two) *relation* instances as arguments and returns a *relation* instance as the result (*makes the algebra "closed"*)
- Relational algebra is a procedural query language
  - Defines a step-by-step procedure for computing the answer

Relational Algebra

- Relational Algebra consists of several groups of operations
  - Unary Relational Operations
    - SELECT (symbol: σ (sigma))
    - PROJECT (symbol: π (pi))
    - RENAME (symbol: ρ (rho))
  - Relational Algebra Operations From Set Theory
    - UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE (or MINUS, **–**)
    - CARTESIAN PRODUCT (**x**)
  - Binary Relational Operations
    - JOIN (several variations of JOIN exist)
    - DIVISION
  - Additional Relational Operations
    - OUTER JOINS
    - AGGREGATE FUNCTIONS
- Each operation returns a relation, and operations can be composed!

SELECT (σ)



- The SELECT operation (denoted by σ(sigma)) is used to select a *subset* of the tuples from a relation based on a ***selection condition***
  - The selection condition acts as a ***filter***
  - Keeps only those tuples that satisfy the qualifying condition
- The SELECT operation is denoted by $\sigma_{<\text{selection condition}>}(R)$ where
  - The symbol σ (sigma) is used to denote the *select* operator
  - The *selection condition* is a Boolean (conditional) expression specified on the attributes of relation R
  - Tuples that make the condition **true** are selected
  - Tuples that make the condition **false** are filtered out

SELECT (σ)

- <selection condition> is a Boolean combination (an expression using logical connectives $\wedge$ and $\vee$) of terms:
  - Condition have the form: **Term** *op* **Term**
    - **Term** is an attribute name or a constant
    - *op* is one of $<$, $\leq$, $=$, $\neq$, $>$, $\geq$
  - Different conditions can be linked together with a boolean expression
    - **(C1** $\wedge$ **C2)**, **(C1** $\vee$ **C2)**, **(**$\neg$**C1)** are conditions where C1 and C2 are conditions
      - $\wedge$ means AND
      - $\vee$ means OR
      - $\neg$ means NOT

Example: SELECT (σ)

Relation R

a	b	c	d
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\sigma_{a=b \wedge d>5}(R)$

a	b	c	d
α	α	1	7
β	β	23	10

Example: SELECT (σ)

- Employee

f_name	l_name	id	sex	salary	superid	dno
Joseph	Chan	999999	M	2950	654321	4
Victor	Wong	001100	M	3000	888555	5
Carrie	Kwan	898989	F	2600	654321	4
Joyce	Fong	345345	F	1200	777888	4

- Find all employees who works in department 4 and whose salary is greater than 2500

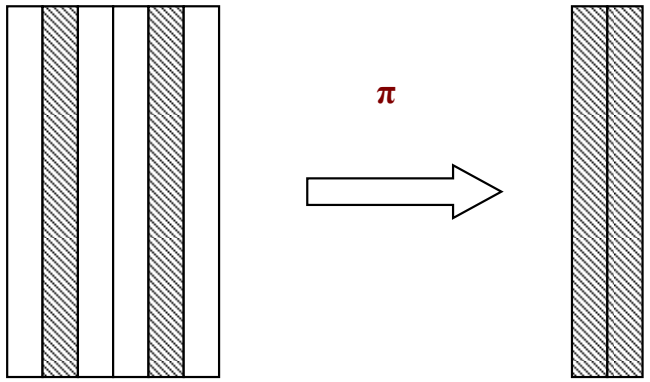
$$\sigma_{dno=4 \wedge salary>2500}(Employee)$$

f_name	l_name	id	sex	salary	superid	dno
Joseph	Chan	999999	M	2950	654321	4
Carrie	Kwan	898989	F	2600	654321	4

SELECT (σ) Properties

- The SELECT operation $\sigma_{<\text{selection condition}>}(R)$ produces a relation S that has the same schema (same attributes) as R
- SELECT σ is commutative:
  - $\sigma_{<\text{condition1}>}(\sigma_{<\text{condition2}>}(R)) = \sigma_{<\text{condition2}>}(\sigma_{<\text{condition1}>}(R))$
- Because of commutativity property, a cascade (sequence) of SELECT operations may be applied in any order:
  - $\sigma_{<\text{cond1}>}(\sigma_{<\text{cond2}>}(\sigma_{<\text{cond3}>}(R)) = \sigma_{<\text{cond2}>}(\sigma_{<\text{cond3}>}(\sigma_{<\text{cond1}>}(R)))$
- A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:
  - $\sigma_{<\text{cond1}>}(\sigma_{<\text{cond2}>}(\sigma_{<\text{cond3}>}(R)) = \sigma_{<\text{cond1}>\wedge<\text{cond2}>\wedge<\text{cond3}>}(R)))$
- The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R

PROJECT (π, Π)



- PROJECT operation is denoted by π (pi), this operation keeps certain *columns* (attributes) from a relation and discards the other columns
  - PROJECT creates a vertical partitioning
- The general form of the *project* operation is:

$$\pi_{<\text{attribute list}>}(R)$$

  - π (pi) is the symbol used to represent the *project* operation
  - <attribute list> is the desired list of attributes from relation R
- The project operation *removes any duplicate tuples*
  - This is because the result of the *project* operation must be a *set of tuples*

PROJECT $\pi_L(R)$

- The projection operator π allows us to extract attributes from a relation
  - Deletes attributes that are not in projection list L
  - Schema of result contains exactly the fields in the projection list, with the same names that they had in the (only) input relation

Relation R

a	b	c
α	10	1
α	20	1
β	30	1
β	40	2

a	c
α	1
α	1
β	1
β	2

$\pi_{a,c}(R)$

Eliminate Duplicated rows

a	c
α	1
β	1
β	2

Example: PROJECT $\pi_L(\mathbf{R})$

1. Find the employee names and department number of all employees
2. Find the sex and department number of all employees

f_name	l_name	id	sex	salary	superid	dno
Joseph	Chan	999999	M	2950	654321	4
Victor	Wong	001100	M	3000	888555	5
Carrie	Kwan	898989	F	2600	654321	4
Joyce	Fong	345345	F	1200	777888	4

$\pi_{lname,fname,dno}(Employee)$

f_name	l_name	dno
Joseph	Chan	4
Victor	Wong	5
Carrie	Kwan	4
Joyce	Fong	4

$\pi_{sex,dno}(Employee)$

sex	dno
M	4
M	5
F	4

PROJECT (π, Π) Properties

- The number of tuples in the result of projection $\pi_{<list>}(R)$ is *always less or equal to* the number of tuples in R
  - If the list of attributes includes a *key* of R, then the number of tuples in the result of PROJECT is *equal* to the number of tuples in R

Relational Algebra Expressions

- We may want to apply several relational algebra operations one after the other
  - Either we can write the operations as a single relational algebra expression by nesting the operations, or
  - We can apply one operation at a time and create intermediate result relations.
- In the latter case, we must give *names* to the relations that hold the intermediate results
- For example: $\pi_{lname,fname}(\sigma_{dno=5}(EMPLOYEE))$, we can apply one operation at a time:
  - DEP5_EMPS $\leftarrow \sigma_{dno=5}(EMPLOYEE)$
  - RESULT $\leftarrow \pi_{lname,fname}(DEP5\_EMPS)$

Example: Compositing Operations

- Substituting an expression where a relation is expected
- The result of an relational-algebra expression is always a relation.

f_name	l_name	id	sex	salary	superid	dno
Joseph	Chan	999999	M	29500	654321	4
Victor	Wong	001100	M	30000	888555	5
Carrie	Kwan	898989	F	26000	654321	4
Joyce	Fong	345345	F	12000	777888	4

f_name	l_name
Joseph	Chan
Carrie	Kwan

$$\pi_{lname,fname}(\sigma_{dno=4 \wedge salary>25000}(Employee))$$

OR

$$R1 \leftarrow \sigma_{dno=4 \wedge salary>25000}(Employee)$$
$$\pi_{lname,fname}(R1)$$

RENAME (ρ)

- The RENAME operator is denoted by ρ (rho)
- Allows to name and therefore to refer to the result of relational algebra expression.
- Allows to refer to a relation by more than one name (e.g., if the same relation is used twice in a relational algebra expression)
- In some cases, we may want to *rename* the attributes of a relation or the relation name or both
  - Useful when a query requires multiple operations
  - Necessary in some cases (see JOIN operation later)

RENAME (ρ)

- The general RENAME operation ρ can be expressed by:
  - $\rho(R(A_1 \rightarrow B_1, A_2 \rightarrow B_2, \ldots, A_n \rightarrow B_n), E)$ or $\rho(R, E)$, with $E(A_1, \ldots, A_n)$, changes both:
    - the relation name to R, *and*
    - the column (attribute) names to $B_1, \ldots, B_n$
    - the position number *i*-th can also be used
    - the resulting relation of $\rho(R, E)$ is R
    - $R \leftarrow E$ is same as $\rho(R, E)$

Example: RENAME (ρ)

- Renaming by attribute name:
  - $\rho(C(sid \rightarrow identity), E)$
  - We may rename more attributes:
  - $\rho(C(sid \rightarrow identity, child \rightarrow dependent), E)$
- Renaming by attribute position:
  - $\rho(C(3 \rightarrow identity), E)$
  - Result is C
  - The 3rd attribute in E is renamed as "*identity*" in C

Example: RENAME (ρ)

$$\rho(R(A_1 \rightarrow B_1, A_2 \rightarrow B_2, \ldots, A_n \rightarrow B_n), E)$$

- The new relation R has the same instance as E, but its schema has attribute B_i instead of attribute A_i
- Necessary if we need to perform a cartesian product or join of a table with itself

ρ(Staff(Name $\rightarrow$ Family_Name, Salary $\rightarrow$ Gross_salary), Employee)

Employee

Name	Salary	Emp_No
Clark	150000	1006
Gates	5000000	1005
Jones	50000	1001
Peters	45000	1002
Phillips	25000	1004
Rowe	35000	1003
Warnock	500000	1007

Staff

Family_Name	Gross_Salary	Emp_No
Clark	150000	1006
Gates	5000000	1005
Jones	50000	1001
Peters	45000	1002
Phillips	25000	1004
Rowe	35000	1003
Warnock	500000	1007

Outline

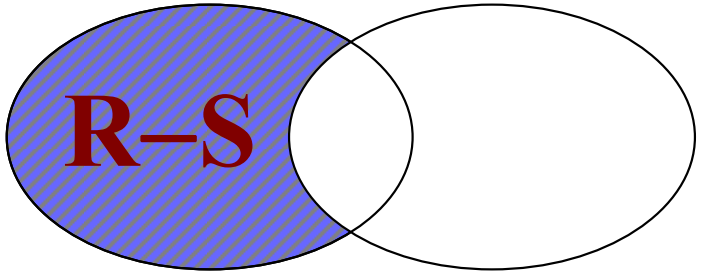
- **Relational Algebra**
  - Unary Relational Operations
  - **Binary Relational Operations**
  - Additional Relational Operations
- Introduction to Relational Calculus
  - Tuple Relational Calculus
  - Domain Relational Calculus

Operations from Set Theory

- Type Compatibility of operands is required for the binary set operation UNION $\cup$, (also for INTERSECTION $\cap$, and SET DIFFERENCE –)
- R1(A_1, A_2, ..., A_n) and R2(B_1, B_2, ..., B_n) are type compatible if:
  - They have *the same number of attributes*, and
  - The domains of corresponding attributes are *type compatible* (i.e. dom(A_i) = dom(B_i) for i = 1, 2, ..., n)
- The resulting relation for R1 $\cup$ R2 (also for R1 $\cap$ R2, or R1 –R2) has the same attribute names as the first operand relation R1 (by convention)

SET DIFFERENCE (–)

- SET DIFFERENCE (also called MINUS or EXCEPT) is denoted by –
- The result of R – S, is a relation that includes all tuples that are in R but not in S
- The two operand relations R and S must be “type compatible”
- Example: R-S



SET DIFFERENCE (–)

R

A	B	C
3	6	7
2	5	7
7	2	3
4	4	3

S

A	B	C
3	4	5
7	2	3

R – S

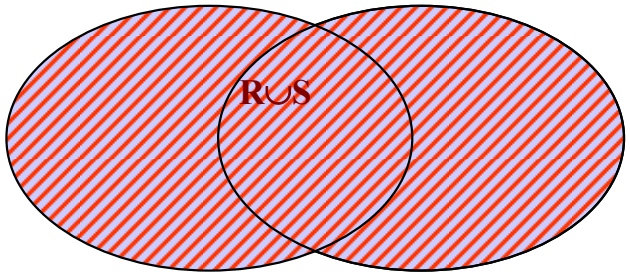
A	B	C
3	6	7
2	5	7
4	4	3

S – R

A	B	C
3	4	5

UNION (∪)

- Binary operation UNION, denoted by ∪
- The result of R ∪ S, is a relation that includes all tuples that are either in R or in S or in both R and S
- Duplicate tuples are eliminated
- The two operand relations R and S must be “type compatible”
- Example: **R** ∪ **S**



UNION (∪)

R

A	B	C
3	6	7
2	5	7
7	2	3
4	4	3

S

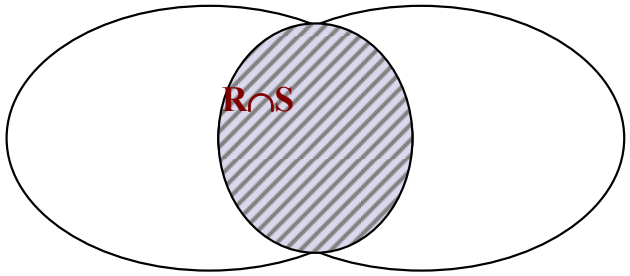
A	B	C
3	4	5
7	2	3

R∪ S

A	B	C
3	6	7
2	5	7
7	2	3
4	4	3
3	4	5

INTERSECTION (∩)

- The result of the operation R ∩ S, is a relation that includes all tuples that are in both R and S
- Intersection is not considered a basic operation, as it can be derived from the basic operations: R ∩ S = R – (R – S)
- The two operand relations R and S must be “type compatible”
- Example: **R ∩ S**



INTERSECTION (∩)

R

A	B	C
3	6	7
2	5	7
7	2	3
4	4	3

S

A	B	C
3	4	5
7	2	3

R∩S

A	B	C
7	2	3

CARTESIAN PRODUCT (×)

- CARTESIAN (or CROSS) PRODUCT Operation is used to combine tuples from two relations in a combinatorial fashion
  - Denoted by $R(A_1, A_2, \ldots, A_n) \times S(B_1, B_2, \ldots, B_m)$
  - Result is a relation Q with degree n + m attributes:
    - $Q(A_1, A_2, \ldots, A_n, B_1, B_2, \ldots, B_m)$, in that order
  - The resulting relation state has one tuple for each combination of tuples - one from R and one from S
  - Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R \cdot n_S$ tuples
- Generally, CROSS PRODUCT is not a meaningful operation
  - Can become meaningful when followed by other operations

CARTESIAN PRODUCT (×)

- R × S returns a relation instance whose schema contains all the attributes of R followed by all the attributes of S
- To keep only combinations of tuples meaningful, typically, we add a SELECT operation after Cartesian product

R

a	b
α	1
β	2

S

c	d	e
α	10	+
β	10	+
β	20	-
γ	10	-

R × S

a	b	c	d	e
α	1	α	10	+
α	1	β	10	+
α	1	β	20	-
α	1	γ	10	-
β	2	α	10	+
β	2	β	10	+
β	2	β	20	-
β	2	γ	10	-

$\sigma_{a=c}(R \times S)$

a	b	c	d	e
α	1	α	10	+
β	2	β	10	+
β	2	β	20	-

JOIN ($\bowtie_{\text{<join condition>}}$)

- This operation is *very important* for any relational database with more than a single relation, because it allows us *combine related tuples* from various relations
- The general form of a join operation on two relations $R(A_1, A_2, \ldots, A_n)$ and $S(B_1, B_2, \ldots, B_m)$ is:

$$R \bowtie_{\text{<join condition>}} S = \sigma_{\text{<join condition>}}(R \times S)$$

where R and S can be any relations that result from general *relational algebra expressions*

EQUIJOIN

- EQUIJOIN Operation is the most common use of join involves join conditions with equality comparisons only
- Such a join, where the only comparison operator used is =, is called an EQUIJOIN
  - In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple
  - The JOIN seen in the previous example was an EQUIJOIN

Example: EQUIJOIN

- A condition join where the condition contains only equalities.
- The conditions of the join have the form R.B = S.B, where we want to join R with S

R

A	B	C
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

B	E
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

R ⋈ S

R.B=S.B

A	R.B	C	S.B	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2

NATURAL JOIN (⋈)

- NATURAL JOIN (⋈), is a further special case of the join operation, which is an EQUIJOIN in which equalities are specified on *all fields having the same name* in R and S
  - The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, *have the same name* in both relations
  - The result is guaranteed not to have two attributes with the same name
- We can simply omit the join condition

Example: NATURAL JOIN

R

A	*B*	*C*
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

S

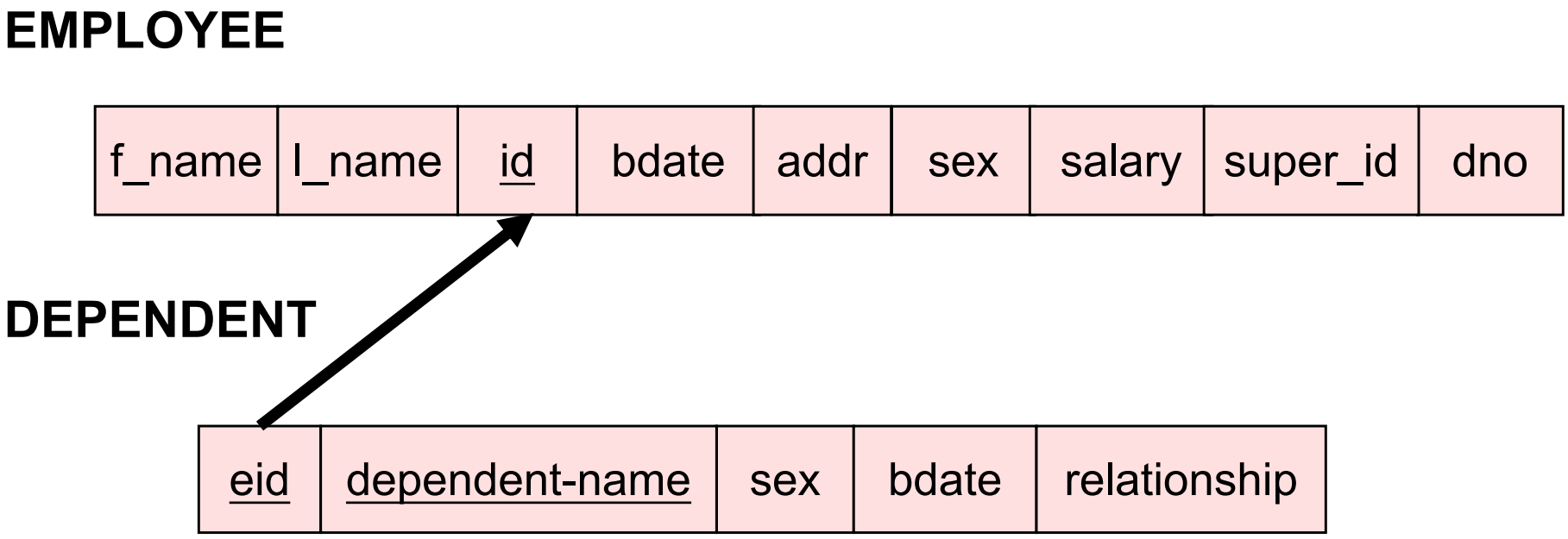
B	*E*
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

R ⋈ S

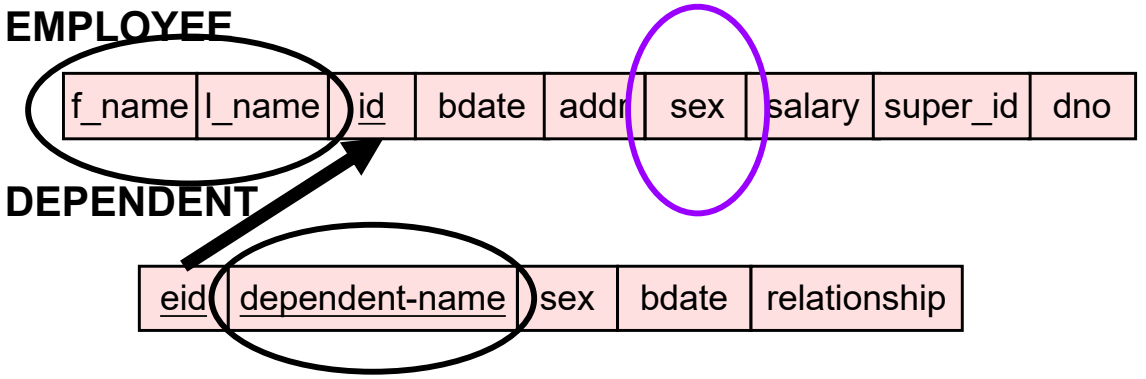
A	*B*	*C*	*E*
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2

Example: Complex Query

- Let us try a more complex query
- Query: Retrieve for each ***female employee*** a list of the ***names*** of her ***dependents***



Example:



- Query: Retrieve for each

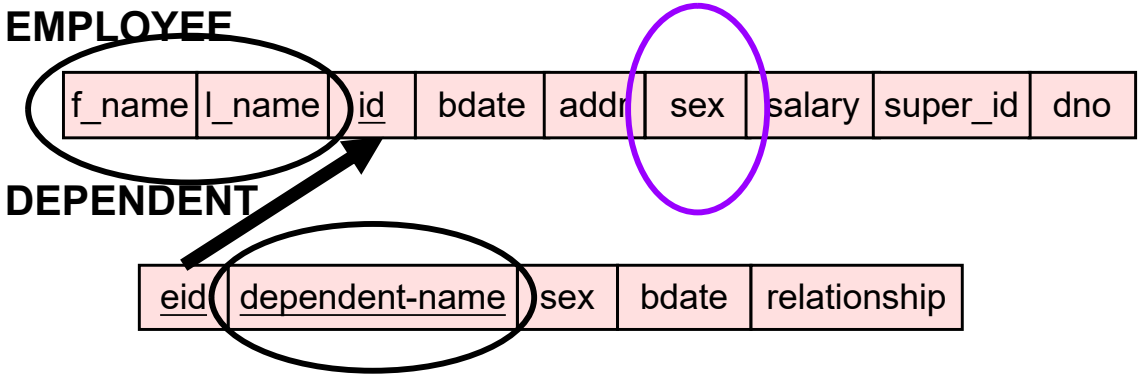
  female employee a list of the ***names*** of her ***dependents***

- Step1 (Female Emps): FemaleEmps $\leftarrow \sigma_{sex='F'}$ (*Employee*)
- Step2 (Their IDs): EmpNames $\leftarrow \pi_{fname,lname,id}$(FemaleEmps)

FemaleEmps

f_name	l_name	id	bdate	address	sex	salary	superid	dno
Alicia	Chan	998877	2-Jul-70	231, Cai Road, HK	F	9500	654321	4
Jennifer	Wong	654321	20-June-60	342, Cheung Road, HK	F	30000	888555	4
Joyce	Fong	345345	19-Dec-80	23, Young Road, HK	F	12000	777888	5

Example:



- Query: Retrieve for each

female employee a list of the ***names*** of her ***dependents***.

- Step3 (Their dependents): EmpNames $\bowtie_{id=eid}$ Dependents

EmpNames

f_name	l_name	id
Alicia	Chan	998877
Jennifer	Wong	654321
Joyce	Fong	345345

Dependents

eid	dep_name	sex	bdate	relationship
334455	Alice	F	5-Apr-90	Daughter
334455	Theodore	M	3-Mar-92	Son
654321	Abner	M	29-Feb-94	Son
123456	Alice	F	2-Nov-97	Daughter

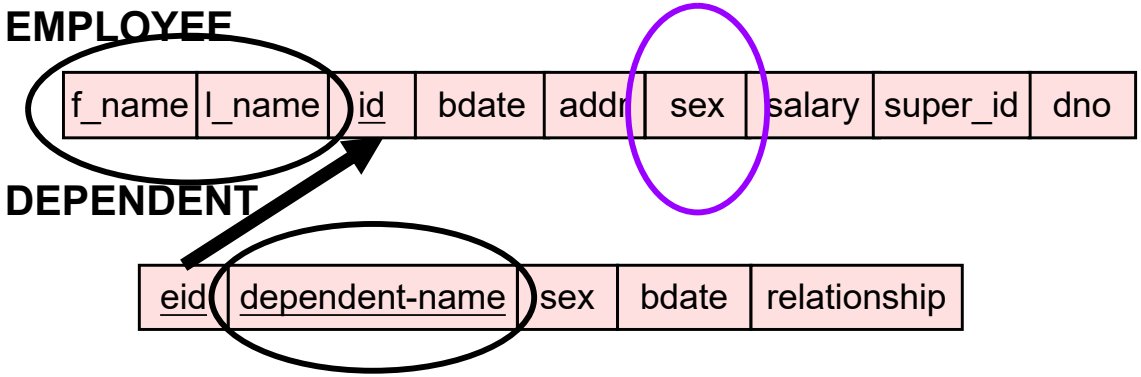
$$ActualDependents \leftarrow EmpNames \bowtie_{id=eid} Dependents$$

$$= \sigma_{id=eid}(EmpNames \times Dependents)$$

FEmpNamesID × Dependents

f_name	l_name	id	eid	dep_name	sex	bdate	relationship
Alicia	Chan	998877	334455	Alice	F	5-Apr-90	Daughter
Alicia	Chan	998877	334455	Theodore	M	3-Mar-92	Son
Alicia	Chan	998877	654321	Abner	M	29-Feb-94	Son
Alicia	Chan	998877	123456	Alice	F	2-Nov-97	Daughter
Jennifer	Wong	654321	334455	Alice	F	5-Apr-90	Daughter
Jennifer	Wong	654321	334455	Theodore	M	3-Mar-92	Son
Jennifer	Wong	654321	654321	Abner	M	29-Feb-94	Son
Jennifer	Wong	654321	123456	Alice	F	2-Nov-97	Daughter
Joyce	Fong	345345	334455	Alice	F	5-Apr-90	Daughter
Joyce	Fong	345345	334455	Theodore	M	3-Mar-92	Son
Joyce	Fong	345345	654321	Abner	M	29-Feb-94	Son
Joyce	Fong	345345	123456	Alice	F	2-Nov-97	Daughter

Example:



- Query: Retrieve for each

  female employee a list of the ***names*** of her ***dependents***.

- Step4: $\pi_{fname,lname,dep\_name}(ActualDependents)$

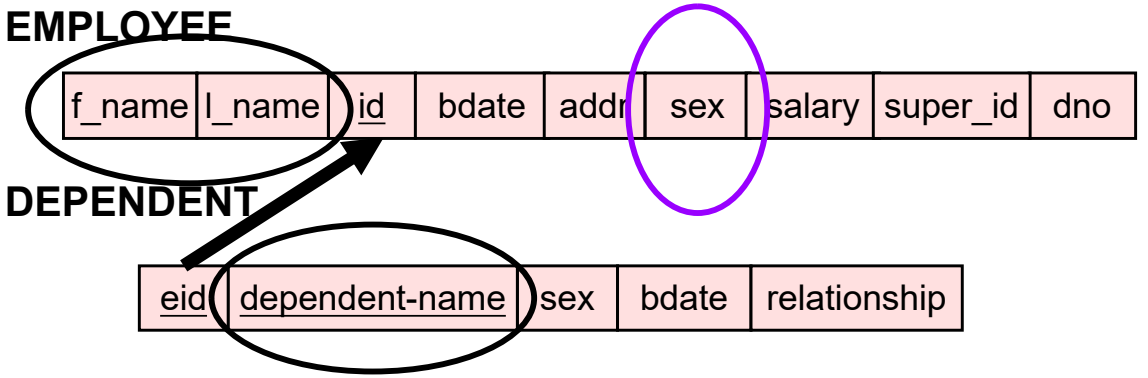
ActualDependents

f_name	l_name	id	eid	dep_name	sex	bdate	relationship
Jennifer	Wong	654321	654321	Abner	M	29-Feb-94	Son

Result

f_name	l_name	dep_name
Jennifer	Wong	Abner

Example:



- Query: Retrieve for each

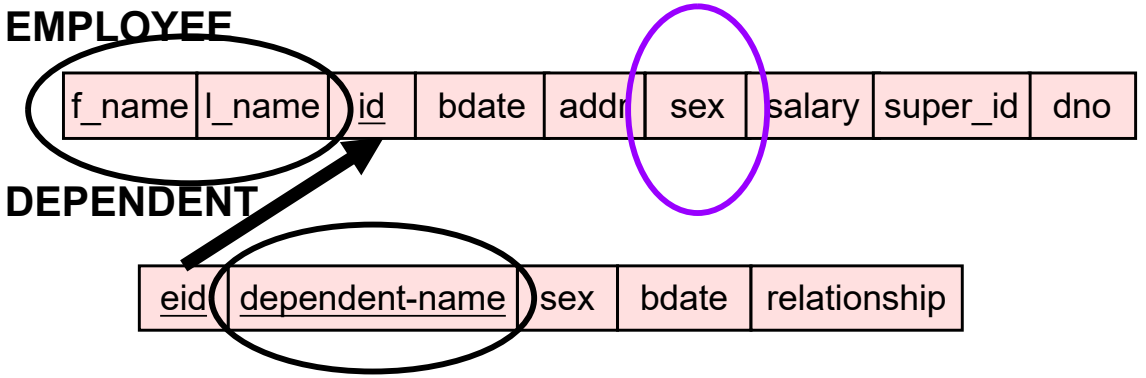
  female employee a list of the ***names*** of her ***dependents***.

- Step1: FemaleEmps $\leftarrow \sigma_{sex='F'}$ (*Employee*)
- Step2: EmpNames $\leftarrow \pi_{fname,lname,id}$(FemaleEmps)
- Step3: *ActualDependents* $\leftarrow$ *EmpNames* $\bowtie_{id=eid}$ *Dependents*
- Step4: $\pi_{fname,lname,dep\_name}$(*ActualDependents*)

Result

f_name	l_name	dep_name
Jennifer	Wong	Abner

Example:



- Query: Retrieve for each

  female employee a list of the ***names*** of her ***dependents***.

- Another Solution:

$$\pi_{fname,lname,dep\_name}\,(\sigma_{sex='F'}\,(Employee \bowtie_{id=eid} Dependents))$$

Result

f_name	l_name	dep_name
Jennifer	Wong	Abner

Condition Join (θ-Join)

- The general case of JOIN operation is called a θ-join:

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

- The join condition is called *theta* (θ).
- *Theta* (θ) can be any general boolean expression on the attributes of R and S; for example:
  - $R.A_i \neq S.A_i$
  - $R.A_i < S.A_i \ \vee (R.A_i = S.A_i \wedge R.A_j < S.A_j)$ ..

Condition Join (θ-Join)

- The general case of JOIN operation is called a θ-join:

$$R \bowtie_\theta S = \sigma_\theta (R \times S)$$

- Simple Example:

S

sid	sname	rating	age
22	Dustin	7	45.0
31	Lubber	8	55.5
58	Rusty	10	35.5

R

sid	bid	day
22	101	10/10/96
58	103	11/12/96

$$S \bowtie_{S.sid<R.sid} R$$

S.sid	sname	rating	age	R.sid	bid	day
22	Dustin	7	45.0	58	103	11/12/96
31	Lubber	8	55.5	58	103	11/12/96

Example: θ-Join

- We have a Flight table that records the Flight Number, Origin, Destination, Departure Time and Arrival Time.
- We join this table with itself (*self-join*) using the condition:
- Flight1.Dest = Flight2.Origin ∧ Flight1.Arr_Time < Flight2.Dept_Time
- What should we get?

Flight1

Num	Origin	Dest	Dep_Time	Arr_Time
334	ORD	MIA	12:00	14:14
335	MIA	ORD	15:00	17:14
336	ORD	MIA	18:00	20:14
337	MIA	ORD	20:30	23:53
394	DFW	MIA	19:00	21:30
395	MIA	DFW	21:00	23:43

Flight2

Num	Origin	Dest	Dep_Time	Arr_Time
334	ORD	MIA	12:00	14:14
335	MIA	ORD	15:00	17:14
336	ORD	MIA	18:00	20:14
337	MIA	ORD	20:30	23:53
394	DFW	MIA	19:00	21:30
395	MIA	DFW	21:00	23:43

Example: θ-Join

- We have a Flight table that records the Flight Number, Origin, Destination, Departure Time and Arrival Time.

$$\text{Flight1.Dest} = \text{Flight2.Origin} \wedge \text{Flight1.Arr\_Time} < \text{Flight2.Dept\_Time}$$

Flight1.Num	Flight1.Origin	Flight1.Dest	Flight1.Dep_Time	Flight1.Arr_Time	Flight2_1.Num	Flight2.Origin	Flight2.Dest	Flight2.Dep_Time	Flight2.Arr_Time
334	ORD	MIA	12:00	14:14	335	MIA	ORD	15:00	17:14
335	MIA	ORD	15:00	17:14	336	ORD	MIA	18:00	20:14
336	ORD	MIA	18:00	20:14	337	MIA	ORD	20:30	23:53
334	ORD	MIA	12:00	14:14	337	MIA	ORD	20:30	23:53
336	ORD	MIA	18:00	20:14	395	MIA	DFW	21:00	23:43
334	ORD	MIA	12:00	14:14	395	MIA	DFW	21:00	23:43

Complete Set of Relational Operations

- The set of operations including SELECT (σ), PROJECT (π), UNION ($\cup$), DIFFERENCE ($-$), and CARTESIAN PRODUCT ($\times$) is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.
- For example:
  - $R \cap S = R - (R - S)$
  - $R \bowtie_{\text{<join condition>}} S = \sigma_{\text{<join condition>}}(R \times S)$

DIVISION

- The division operation is useful for expressing certain kinds of queries, for example, “find the names of student who have passed all courses.”
- Consider two relations A and B
  - $A_{\{x, y\}}$ has exactly two attributes x and y
  - $B_{\{y\}}$ has just one attribute y, with the same domain as in A
  - A/B contains all x tuples, such that for every y tuple in B there is a <x,y> tuple in A

A

x	y
s1	p2
s1	p3
s1	p4
s2	p2
s3	p2
s4	p2
s4	p4

/

B

y
p2
p4

=

A/B

x
s1
s4

DIVISION

A

x	y
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
ε	1
ε	2

B

y
1
2

A / B

x
α
ε

- Another way to understand division:
- For each x value in A, consider the set of y values that appear in records of A with that x value
- If this set contains all y values in B, the x value is in the result of A/B

Example: DIVISION

- Find all student IDs (sids) of the students who took ***all*** courses in table Course

Take

sid	cid
1	231
1	170
1	111
1	001
2	231
2	001
3	170
4	231
4	170

/

Course

cid
231
170

=

Take / Course

sid
1
4

Example: DIVISION

- Find all student IDs (sids) of the students who took ***all*** courses provided by ***CSE*** department

?

Take

sid	cid
1	231
1	170
1	111
1	001
2	231
2	001
3	170
4	231
4	170

Course

cid	dept
231	CSE
170	CSE
001	LANG
111	ECE
123	ECE

More Examples

Sailors (sid, sname, age)

Boats (bid, bname, color)

Reserves (sid, bid, date)

- Consider the above schemas, the primary key fields are underlined.

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 1: Find the names of sailors who have reserved boat with bid = 103

?

?

?

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 2: Find the names of sailors who have reserved at least a red boat
  - Solution 1:

    π_{sname} (($\sigma_{color='red'}$ Boats) $\bowtie$ Reserves $\bowtie$Sailors)

  - Solution 2:

    π_{sname} (π_{sid} ((π_{bid} $\sigma_{color='red'}$ Boats) $\bowtie$Reserves) $\bowtie$Sailors)

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 3: Find the names of sailors who have reserved at least a red or a green boats
  - We can identify all red or green boats, then find sailors who have reserved one of these boats

  — Solution 1:

  ρ (Tempboats, (σ $_{color='red' \vee color='green'}$ Boats))

  π $_{sname}$ (Tempboats $\bowtie$ Reserves $\bowtie$ Sailors)

  What happens if $\vee$ is replaced by $\wedge$ in this query?

  — Solution 2:

  π $_{sname}$ ((σ $_{color='red'}$ Boats) $\bowtie$ Reserves) $\bowtie$ Sailors)

  $\cup$ π $_{sname}$ ((σ $_{color='green'}$ Boats) $\bowtie$ Reserves) $\bowtie$ Sailors)

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 4: Find the names of sailors who have reserved at least a red boat and at least a green boat
  - The previous Solution 1 would not work
  - We must identify sailors who have reserved red boats, sailors who have reserved green boats, then find the intersection
  - Note that sid is a key for Sailors

  — Solution:

  ρ(Tempred, π_{sid} (($\sigma_{color='red'}$ Boats) $\bowtie$ Reserves))

  ρ(Tempgreen, π_{sid} (($\sigma_{color='green'}$ Boats) $\bowtie$ Reserves))

  π_{sname} ((Tempred $\cap$ Tempgreen) $\bowtie$ Sailors)

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 5: Find the names of sailors who have reserved all boats
  - Solution:

    ρ(Tempsids, ($\pi_{sid,bid}$ Reserves) / (π_{bid} Boats))

    π_{sname} (Tempsids $\bowtie$ Sailors)

  What if we simply do, Reserves / (π_{bid} Boats) ?

date	sid	bid
2-3-2002	007	A
3-7-2002	007	B

Returns nothing,
When we expect 007

date	sid	bid
2-3-2002	007	A
2-3-2002	007	B

Returns (2-3-2002, 007)

More Examples

Sailors (sid, sname, age)
Boats (bid, bname, color)
Reserves (sid, bid, date)

- Query 6: Find the names of sailors who have reserved all red boats:

  - Solution:

    ρ(Tempsids, ($\pi_{sid,bid}$ Reserves) / (π_{bid} ($\sigma_{color='red'}$ Boats))

    π_{sname} (Tempsids $\bowtie$ Sailors)

Summary

- The relational model has rigorously defined query languages that are simple and powerful
- Relational algebra is operational; useful as an internal representation for query evaluation steps
- Typically there are several ways of expressing a given query; a query optimizer should choose an efficient version